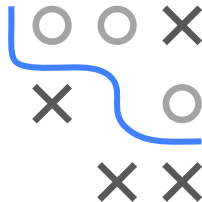


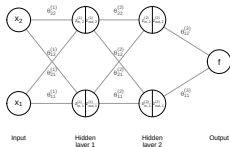
Optimization in Machine Learning

Advanced first order methods Backpropagation and autodiff



Learning goals

- DNNs as computational graphs
- Gradients via the chain rule and vector-Jacobian products (VJPs)
- Backpropagation algorithm
- Implementation via automatic differentiation



Slides based on

► The JAX authors 2024

► Dagr  ou et al. 2024

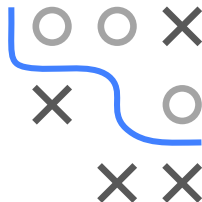
COMPUTING GRADIENTS FOR DNNs

- We now train **deep neural networks (DNNs)**: models with a very large parameter vector $\theta \in \mathbb{R}^d$ (d potentially in the millions)
- All methods discussed in this chapter rely on the gradient of the (scalar) empirical risk w.r.t. θ :

$$\nabla_{\theta} \mathcal{R}_{\text{emp}}(\theta) \in \mathbb{R}^{1 \times d}$$

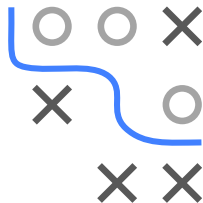
(SGD relies on mini-batch estimates of the gradient)

- For brevity, we write $L := \mathcal{R}_{\text{emp}}(\theta)$ in the following
- For brevity, we write $L := \mathcal{R}_{\text{emp}}(\theta)$ in the following
- **Question:** How to compute it **efficiently**?



DNNs AS COMPUTATIONAL GRAPHS

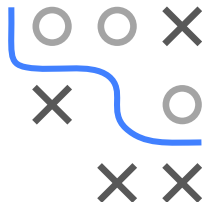
- A DNN can be represented as a **directed acyclic graph** (DAG)
 - An **ordered sequence of nodes** with **directed edges**
 - Nodes hold intermediate values $\mathbf{z}_i$ according to operations $h_i(\cdot)$ of edges feeding into them
- Layer i maps $\mathbf{z}_{i-1}$ to $\mathbf{z}_i = h_i(\mathbf{z}_{i-1}; \theta_i)$
 - e.g. $\mathbf{z}_i = \sigma(\mathbf{W}_i \mathbf{z}_{i-1} + \mathbf{b}_i)$ with $\theta_i = (\mathbf{W}_i, \mathbf{b}_i)$
- A DNN can be represented as a **directed acyclic graph** (DAG)
 - An **ordered sequence of nodes** with **directed edges**
 - Nodes hold intermediate values $\mathbf{z}_i$ according to operations $h_i(\cdot)$ of edges feeding into them
- Layer i maps $\mathbf{z}_{i-1}$ to $\mathbf{z}_i = h_i(\mathbf{z}_{i-1}; \theta_i)$
 - e.g. $\mathbf{z}_i = \sigma(\mathbf{W}_i \mathbf{z}_{i-1} + \mathbf{b}_i)$ with $\theta_i = (\mathbf{W}_i, \mathbf{b}_i)$
- A DNN can be represented as a **directed acyclic graph** (DAG):
an ordered sequence of **nodes** with **directed edges**
- Each **node** $\mathbf{z}_i \in \mathbb{R}^{p_i}$, $i = 0, \dots, m$, collects the intermediate values of all p_i neurons of the i -th layer
(The input acts as the first node: $\mathbf{z}_0 = \mathbf{x}$)
- The **directed edge** into node i is the i -th layer: a **function**
mapping the previous node to the next



GRADIENT VIA THE CHAIN RULE

- The risk gradient w.r.t. the parameter vector θ_i of layer i factorizes via the **chain rule**:

$$\begin{aligned}\nabla_{\theta_i} \mathcal{R}_{\text{emp}}(\theta) &= \frac{\partial \mathcal{R}_{\text{emp}}(\theta)}{\partial \mathbf{z}_m} \frac{\partial \mathbf{z}_m}{\partial \mathbf{z}_{m-1}} \cdots \frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{z}_i} \frac{\partial \mathbf{z}_i}{\partial \theta_i} \\ &= \underbrace{\frac{\partial \mathcal{R}_{\text{emp}}(\theta)}{\partial \mathbf{z}_i}}_{=: \bar{\mathbf{z}}_i \in \mathbb{R}^{1 \times p_i}} \underbrace{\frac{\partial \mathbf{z}_i}{\partial \theta_i}}_{\in \mathbb{R}^{p_i \times d_i}} \in \mathbb{R}^{1 \times d_i}\end{aligned}$$



- $\Rightarrow$ A complicated gradient becomes a **product of simple, local derivatives**
- $\Rightarrow$ Each layer's gradient $\nabla_{\theta_i} L$ requires its associated '**node adjoint**' $\bar{\mathbf{z}}_i \in \mathbb{R}^{1 \times p_i}$
- $\Rightarrow$ Each layer's gradient $\nabla_{\theta_i} L$ requires its associated '**node adjoint**' $\bar{\mathbf{z}}_i \in \mathbb{R}^{1 \times p_i}$
- $\Rightarrow$ Each layer's risk gradient $\nabla_{\theta_i} \mathcal{R}_{\text{emp}}(\theta)$ requires its associated '**node adjoint**' $\bar{\mathbf{z}}_i$

ADJOINTS AS SEQUENCES OF VJPS

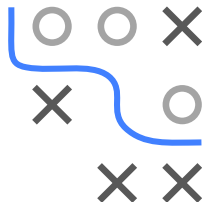
- All node adjoints $\bar{\mathbf{z}}_i$ can be recursively computed in reverse order for $i = m, (m - 1), \dots, 1$:
- Starting from $\bar{\mathbf{z}}_m = \frac{\partial \mathcal{R}_{\text{emp}}(\theta)}{\partial \mathbf{z}_m}$, all remaining node adjoints $\bar{\mathbf{z}}_i$ can be recursively computed in reverse order for $i = m - 1, m - 2, \dots, 1$:

$$\bar{\mathbf{z}}_i = \frac{\partial \mathcal{R}_{\text{emp}}(\theta)}{\partial \mathbf{z}_{i+1}} \frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{z}_i} = \bar{\mathbf{z}}_{i+1} \frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{z}_i}$$

- $\bar{\mathbf{z}}_{i+1} \in \mathbb{R}^{1 \times p_{i+1}}$ is a (row) **vector**
- $\frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{z}_i} \in \mathbb{R}^{p_{i+1} \times p_i}$ is a **Jacobian**

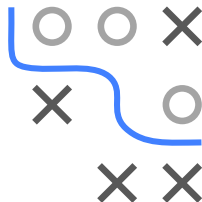
$\Rightarrow$ This constitutes a sequence of **vector-Jacobian products (VJPs)**

$\Rightarrow$ Each $\bar{\mathbf{z}}_i$ must only be computed **once** and can be **reused** for lower layers



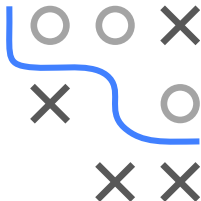
BACKPROPAGATION ALGORITHM

- This motivates the **backpropagation** algorithm to compute the full gradient $\nabla_{\theta} \mathcal{R}_{\text{emp}}(\theta)$ in **two phases**:
- **Forward pass**
 - Set $\mathbf{z}_0 = \mathbf{x}$ and walk forward through the graph up to $\mathcal{R}_{\text{emp}}(\theta)$
 - While doing so, **cache** every node value $\mathbf{z}_i$, because the derivatives in the subsequent phase are evaluated at these values
- **Backward pass**
 - Compute the adjoints $\bar{\mathbf{z}}_i$ as a sequence of VJPs by moving backward through the graph
 - Obtain the risk gradient w.r.t. each layer's parameter vector θ_i as $\nabla_{\theta_i} \mathcal{R}_{\text{emp}}(\theta) = \bar{\mathbf{z}}_i \frac{\partial \mathbf{z}_i}{\partial \theta_i}$ (again a VJP)



AUTOMATIC DIFFERENTIATION

- (Reverse-mode) automatic differentiation (**autodiff**) efficiently automates the backpropagation algorithm
 - Traces code to automatically build a computational graph
 - Decomposes DNNs into **primitive operations** (e.g. $+$, $\cdot$, σ , $\exp$, $\dots$; more granular than entire layers)
(Layer $h_i : \mathbb{R}^{p_{i-1}} \rightarrow \mathbb{R}^{p_i}$ becomes $h_i = h_{i,k} \circ \dots \circ h_{i,1}$ with $h_{i,1} : \mathbb{R}^{p_{i-1}} \rightarrow \mathbb{R}^{p_{i,1}}$, $\dots$, $h_{i,k} : \mathbb{R}^{p_{i,k-1}} \rightarrow \mathbb{R}^{p_i}$)
 - Automatically computes derivatives
 - Applies the chain rule on the (more granular) graph and efficiently computes VJPs in the backward pass via rules (see next slides)
- $\Rightarrow$ Autodiff is a key component powering modern ML frameworks (e.g. autograd in PyTorch)

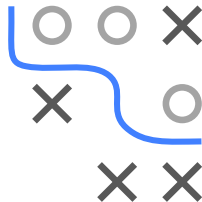


VJP RULES

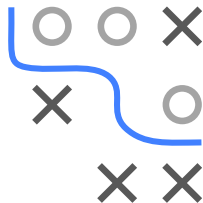
- Consider a primitive operation $g : \mathbb{R}^{q_{\text{in}}} \rightarrow \mathbb{R}^{q_{\text{out}}}$ and the VJP $\mathbf{v} \frac{\partial g(\mathbf{z})}{\partial \mathbf{z}} \in \mathbb{R}^{1 \times q_{\text{in}}}$
 - $\mathbf{v} \in \mathbb{R}^{1 \times q_{\text{out}}}$ as an arbitrary (row) vector
 - $\frac{\partial g(\mathbf{z})}{\partial \mathbf{z}} = \mathbf{J}_g(\mathbf{z}) \in \mathbb{R}^{q_{\text{out}} \times q_{\text{in}}}$ as the Jacobian of the operation evaluated at $\mathbf{z}$

(E.g. $\mathbf{v} = \bar{\mathbf{z}}_i$ may be the adjoint of node $\mathbf{z}_i$ and $g = h_{i,k}$ the last primitive operation of layer i)

- Autodiff stores efficient computation rules for these VJPs, i.e. $\text{vjp}_g(\mathbf{v}, \mathbf{z}, g(\mathbf{z})) = \mathbf{v} \frac{\partial g(\mathbf{z})}{\partial \mathbf{z}}$
 - Reformulation that depends only on $\mathbf{v}$, the operation input $\mathbf{z}$ and (cached) output $g(\mathbf{z})$ $\Rightarrow$ The (potentially large) Jacobian $\mathbf{J}_g(\mathbf{z})$ need not be materialized and is only implicitly multiplied



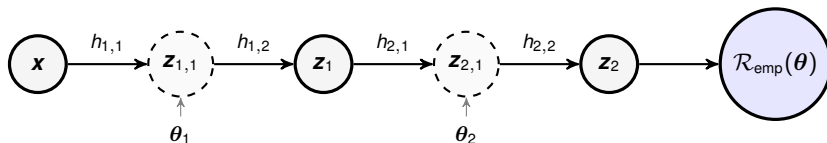
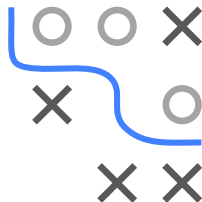
VJP RULES: EXAMPLE



- **Example:** Consider the **elementwise sigmoid** $g = \sigma : \mathbb{R}^q \rightarrow \mathbb{R}^q$
 - i.e. $\mathbf{a} := \sigma(\mathbf{z})$ with $a_j = \sigma(z_j)$, each output depending only on its own input
 - The Jacobian $\mathbf{J}_\sigma(\mathbf{z}) = \frac{\partial \sigma(\mathbf{z})}{\partial \mathbf{z}} \in \mathbb{R}^{q \times q}$ is **diagonal**
 - $[\mathbf{J}_\sigma(\mathbf{z})]_{jj} = \sigma'(z_j) = a_j(1 - a_j)$ and $[\mathbf{J}_\sigma(\mathbf{z})]_{jk} = 0$ for $j \neq k$
 - Thus the VJP rule collapses to an elementwise product:
 - $\text{vjp}_\sigma(\mathbf{v}, \mathbf{z}, \mathbf{a}) = \mathbf{v} \frac{\partial \sigma(\mathbf{z})}{\partial \mathbf{z}} = \mathbf{v} \odot \mathbf{a}^\top \odot (1 - \mathbf{a})^\top$
- $\Rightarrow \mathcal{O}(q^2)$ cost for building, storing and multiplying the explicit Jacobian drops to $\mathcal{O}(q)$

WORKED EXAMPLE: FORWARD PASS

- Consider a two-layer DNN with $\theta = (\theta_1, \theta_2)$, $\theta_i = (\mathbf{W}_i, \mathbf{b}_i)$, $\mathbf{W}_i \in \mathbb{R}^{p_i \times p_{i-1}}$, $\mathbf{b}_i \in \mathbb{R}^{p_i}$, scalar output ($p_2 = 1$) and squared loss $\mathcal{R}_{\text{emp}}(\theta) = \frac{1}{2}(\mathbf{z}_2 - y)^2$ for one observation $(\mathbf{x}, y)$
- Each layer consists of two primitive operations, $h_i = h_{i,2} \circ h_{i,1}$:
 - Affine map $h_{i,1}(\mathbf{z}_{i-1}) = \mathbf{W}_i \mathbf{z}_{i-1} + \mathbf{b}_i$ with intermediate value $\mathbf{z}_{i,1} \in \mathbb{R}^{p_i}$
 - Elementwise sigmoid $h_{i,2} = \sigma$



WORKED EXAMPLE: BACKWARD PASS

- **Adjoint**s via VJPs in reverse order:

$$\bar{\mathbf{z}}_2 = \frac{\partial \mathcal{R}_{\text{emp}}(\boldsymbol{\theta})}{\partial \mathbf{z}_2} = \mathbf{z}_2 - y \quad \in \mathbb{R}$$

$$\bar{\mathbf{z}}_{2,1} = \bar{\mathbf{z}}_2 \frac{\partial \mathbf{z}_2}{\partial \mathbf{z}_{2,1}} = \bar{\mathbf{z}}_2 \mathbf{z}_2 (1 - \mathbf{z}_2) \quad \in \mathbb{R}$$

$$\bar{\mathbf{z}}_1 = \bar{\mathbf{z}}_{2,1} \frac{\partial \mathbf{z}_{2,1}}{\partial \mathbf{z}_1} = \bar{\mathbf{z}}_{2,1} \mathbf{W}_2 \quad \in \mathbb{R}^{1 \times p_1}$$

$$\bar{\mathbf{z}}_{1,1} = \bar{\mathbf{z}}_1 \frac{\partial \mathbf{z}_1}{\partial \mathbf{z}_{1,1}} = \bar{\mathbf{z}}_1 \odot \mathbf{z}_1^\top \odot (1 - \mathbf{z}_1)^\top \quad \in \mathbb{R}^{1 \times p_1}$$

- **Risk gradients** w.r.t. $\boldsymbol{\theta}_i = (\mathbf{W}_i, \mathbf{b}_i)$, again via VJPs (with $\mathbf{z}_0 = \mathbf{x}$):

$$\nabla_{\mathbf{b}_i} \mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = \bar{\mathbf{z}}_{i,1} \frac{\partial \mathbf{z}_{i,1}}{\partial \mathbf{b}_i} = \bar{\mathbf{z}}_{i,1} \quad \in \mathbb{R}^{1 \times p_i}$$

$$\nabla_{\mathbf{W}_i} \mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = \bar{\mathbf{z}}_{i,1} \frac{\partial \mathbf{z}_{i,1}}{\partial \mathbf{W}_i} = \bar{\mathbf{z}}_{i,1}^\top \mathbf{z}_{i-1}^\top \quad \in \mathbb{R}^{p_i \times p_{i-1}} \text{ (shape of } \mathbf{W}_i)$$

